
Chapitre 4

Architecture Globale du Projet

Architecture Technique, Collecte des Données et Pipeline de Traitement

Réalisé par :

Soubaibi Omar Yassine Baizi

Encadré par :

Mme. OUNACER SOUMAYA

Filière : 4IADM G5-A

Année Universitaire : 2025–2026

Table des matières

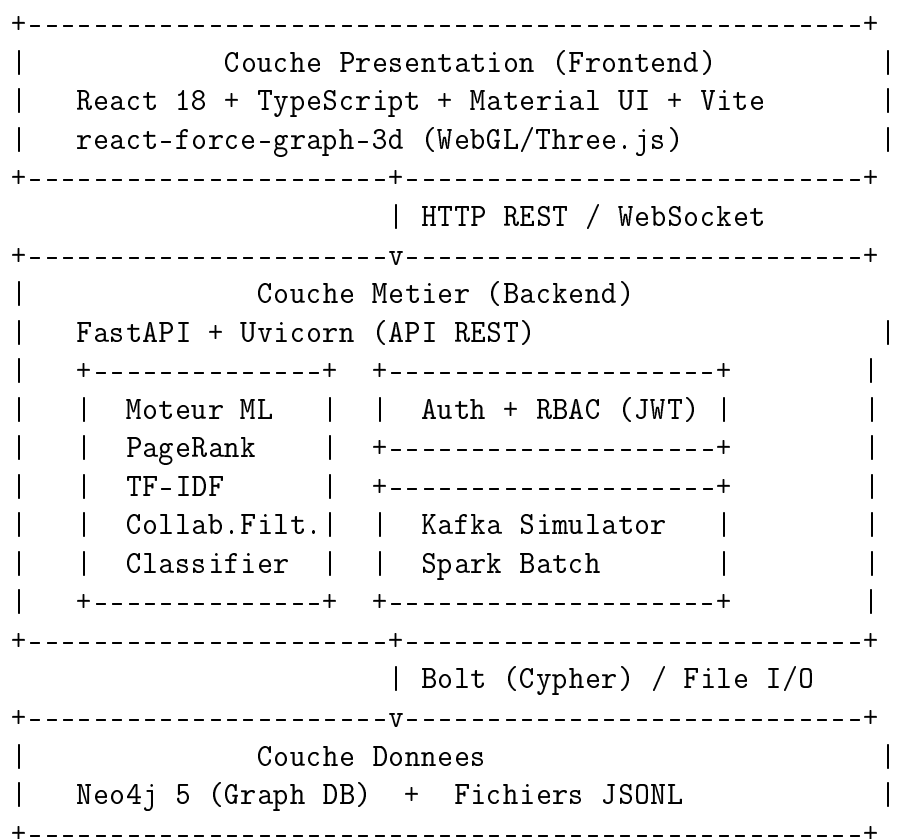
1	Introduction	2
2	Architecture 3-Tiers	2
2.1	Couche Présentation (Frontend)	2
2.2	Couche Métier (Backend)	3
2.3	Couche Données	3
3	Architecture des Flux de Données	4
3.1	Cycle de Vie d'une Requête	4
3.2	Flux de Données Détaillé	4
4	Collecte des Données	5
4.1	Stratégie de Collecte	5
4.2	Sources de Données	5
4.3	Processus de Collecte Actuel	6
5	Pipeline d'Ingestion dans Neo4j	6
5.1	Chargement des Nœuds	6
5.2	Création des Relations	6
5.3	Indexation	7
6	Pipeline Machine Learning	7
6.1	Architecture du Pipeline ML	7
6.2	Détail des Algorithmes	8
7	Pipeline Big Data	8
7.1	Simulateur Kafka	8
7.2	Analyses Spark (Batch)	9
8	Architecture de Déploiement	9
8.1	Conteneurisation Docker	9
8.2	Configuration Nginx	10
8.3	Ordre de Démarrage et Dépendances	10
9	Description des Endpoints API	10
9.1	Endpoints Principaux	10
10	Sécurité et Authentification	11
10.1	Flux d'Authentification JWT	11
10.2	Matrice de Contrôle d'Accès (RBAC)	12
11	Conclusion	12

1. Introduction

Ce chapitre présente l'architecture globale du projet Nexora en détaillant les choix technologiques, l'organisation des composants, les mécanismes de collecte des données et les pipelines de traitement. L'architecture a été conçue pour répondre aux exigences de performance, de scalabilité et de maintenabilité identifiées dans le cahier des charges, tout en intégrant des processus de collecte et de transformation des données adaptés au contexte Big Data du projet.

2. Architecture 3-Tiers

L'application Nexora suit une architecture **3-tiers** séparant strictement la présentation, la logique métier et les données :



2.1 Couche Présentation (Frontend)

La couche présentation est une application **SPA (Single Page Application)** construite avec :

Technologie	Version	Rôle
React	18.x	Bibliothèque UI déclarative à base de composants.
TypeScript	5.x	Superset typé de JavaScript pour la robustesse du code.

Material UI (MUI)	5.x	Système de design avec composants pré-construits et thème personnalisable.
Vite	5.x	Bundler et serveur de développement ultra-rapide (HMR).
react-force-graph-3d	–	Visualisation de graphes 3D interactive via WebGL/Three.js.
Axios	–	Client HTTP pour les appels API REST.
React Router	6.x	Routage SPA côté client.

Fonctionnement : Le frontend est compilé en fichiers statiques (HTML, JS, CSS) par Vite lors du build de production, puis servi par **Nginx Alpine** qui gère également le proxy inverse vers le backend.

2.2 Couche Métier (Backend)

Le backend est le cœur logique de l'application, exposant une **API REST** via FastAPI :

Technologie	Version	Rôle
FastAPI	0.100+	Framework API Python asynchrone, auto-documentation Swagger/OpenAPI.
Uvicorn	–	Serveur ASGI haute performance (HTTP + WebSocket).
Pydantic	2.x	Validation des données et sérialisation avec schémas typés.
bcrypt	–	Hachage sécurisé des mots de passe.
PyJWT	–	Création et validation des tokens d'authentification JWT.
Scikit-learn	–	Moteur ML : TF-IDF, Logistic Regression, similarité cosinus.
Neo4j Driver	–	Driver Python officiel pour la communication Bolt avec Neo4j.
PySpark	–	Traitement analytique par lot des données volumineuses.

2.3 Couche Données

Technologie	Version	Rôle
Neo4j Community	5.x	Base de données orientée graphe pour le stockage et le requêtage des entités et relations.
Plugin APOC	–	Bibliothèque de procédures avancées pour Neo4j (importation en masse, algorithmes de graphe).
Fichiers JSONL	–	Source de données primaire pour les modèles ML et les analyses Spark.

3. Architecture des Flux de Données

3.1 Cycle de Vie d'une Requête

Chaque requête utilisateur suit le chemin suivant :

```
Client (Navigateur)
|
v
Nginx (port 3000)
+-- /api/* -> Proxy -> FastAPI (port 8000)
|                                     +-> Neo4j (Cypher via Bolt)
|                                     +-> Moteur ML (JSONL -> prediction)
|                                     +-> Kafka Simulator / Spark
|
+-- /* -> Fichiers statiques React (SPA fallback)
```

3.2 Flux de Données Détaillé

```
+-----+
|                                     |
|          FLUX DE DONNEES NEXORA   |
|                                     |
+-----+
|                                     |
| [1] COLLECTE                       |
| |                                  |
| | +-- generate_data.py -> employees.jsonl |
| | +-- generate_data.py -> skills.jsonl   |
| | +-- generate_data.py -> projects.jsonl |
| | +-- generate_data.py -> documents.jsonl |
| |                                     |
| [2] INGESTION                     |
| |                                  | |
| | +-- JSONL -> Neo4j (Cypher LOAD)      |
| | |                                   Expert, Skill, Project, Document |
| | |                                   + Relations (HAS_SKILL, WORKS_ON...) |
| | |                                   |
| | +-- JSONL -> Modeles ML (en memoire)  |
| | |                                   TF-IDF vectors, User-Skill matrix |
| |                                     |
| [3] TRAITEMENT                    |
| |                                  | |
| | +-- Temps reel : API REST (FastAPI)   |
| | |                                   +-- Requetes Cypher -> Neo4j -> Reponse JSON |
| | |                                   |
| | +-- Machine Learning : Moteur ML     |
| | |                                   +-- PageRank (graphe biparti) |
| | |                                   +-- Recommandation (TF-IDF + cosinus) |
| | |                                   +-- Filtrage collaboratif (matrice user-skill) |
| | |                                   +-- Classification (TF-IDF + Log. Reg.) |
| |                                     |
```

```

|           |
|         +-- Batch : PySpark
|         +-- Skill Analytics
|         +-- Expert Rankings
|         +-- Department Analytics
|         +-- Project Analytics
|
| [4] PRESENTATION
|
|         +-- Dashboard (graphiques, KPIs)
|         +-- Graphe 3D (WebGL)
|         +-- Resultats de recherche
|         +-- Chatbot Veda (LLM)
|
+-----+

```

4. Collecte des Données

4.1 Stratégie de Collecte

Dans le cadre de Nexora, la collecte des données suit une approche en **deux temps** adaptée au contexte du projet :

1. **Phase 1 — Génération synthétique** : Un générateur de données (`generate_data.py`) produit des jeux de données réalistes simulant une grande organisation. Cette approche permet de tester l'ensemble du pipeline sans dépendre de données réelles.
2. **Phase 2 — Intégration de sources réelles (extensible)** : L'architecture est conçue pour accueillir des connecteurs vers des sources de données d'entreprise (SIRH, LDAP, GitHub, Jira, Slack).

4.2 Sources de Données

Source	Type de données	Format	Fréquence
Générateur Python	Employés, Skills, Projets, Documents	JSONL	À la demande (ponctuel)
SIRH (futur)	Profils, départements, organigramme	API REST	Quotidien
GitHub (futur)	Contributions, langages, repos	API REST	Temps réel (webhooks)
Jira (futur)	Tickets, assignations, compétences déduites	API REST	Horaire
LMS / e-learning	Certifications, formations complétées	API / CSV	Hebdomadaire

4.3 Processus de Collecte Actuel

Le processus de collecte actuel repose sur le script `generate_data.py` qui suit les étapes suivantes :

1. **Initialisation** : Configuration du seed aléatoire et des paramètres d'échelle.
2. **Génération des compétences** : Création du référentiel de 82 compétences réparties en 6 catégories.
3. **Génération des employés** : Création des profils avec attribution réaliste des compétences basée sur l'affinité département-compétence.
4. **Génération des documents** : Création des publications avec attribution aux auteurs (employés existants).
5. **Génération des projets** : Création des projets avec constitution des équipes (échantillonnage parmi les employés).
6. **Sérialisation** : Écriture des données au format JSONL dans le dossier `data/`.

5. Pipeline d'Ingestion dans Neo4j

Une fois les données collectées (fichiers JSONL), elles sont ingérées dans la base de données Neo4j selon le processus suivant :

5.1 Chargement des Nœuds

```
1 // 1. Creation des noeuds Expert
2 LOAD CSV FROM 'file:///employees.jsonl' AS line
3 CREATE (e:Expert {
4     id: line.id,
5     name: line.name,
6     email: line.email,
7     department: line.department,
8     role: line.role,
9     location: line.location,
10    experience_years: line.experience_years
11 })
12
13 // 2. Creation des noeuds Skill
14 LOAD CSV FROM 'file:///skills.jsonl' AS line
15 CREATE (s:Skill {
16     id: line.id,
17     name: line.name,
18     category: line.category,
19     demand: line.demand
20 })
```

5.2 Création des Relations

```

1 // Relations HAS_SKILL (Expert -> Skill)
2 MATCH (e:Expert), (s:Skill)
3 WHERE s.name IN e.skills
4 CREATE (e)-[:HAS_SKILL {level: skill.level}]->(s)
5
6 // Relations WORKS_ON (Expert -> Project)
7 MATCH (e:Expert), (p:Project)
8 WHERE e.id IN p.team_members
9 CREATE (e)-[:WORKS_ON]->(p)
10
11 // Relations AUTHORED (Expert -> Document)
12 MATCH (e:Expert), (d:Document)
13 WHERE e.id = d.author
14 CREATE (e)-[:AUTHORED]->(d)

```

5.3 Indexation

```

1 // Index pour performances de requetage
2 CREATE INDEX expert_id FOR (e:Expert) ON (e.id)
3 CREATE INDEX skill_name FOR (s:Skill) ON (s.name)
4 CREATE INDEX project_id FOR (p:Project) ON (p.id)
5 CREATE INDEX document_id FOR (d:Document) ON (d.id)

```

6. Pipeline Machine Learning

6.1 Architecture du Pipeline ML

```

+-----+
|          PIPELINE ML NEXORA          |
+-----+
|
|  Donnees JSONL
|
|  +--> Pretraitement
|      |
|      |  +-- Nettoyage texte
|      |  +-- Construction matrice user-skill
|      |  +-- Construction graphe biparti
|      |
|      |  +--> Entrainement
|      |      +-- TF-IDF Vectorizer (profils)
|      |      +-- Logistic Regression (documents)
|      |      +-- PageRank iteratif (graphe)
|      |
|      |  +--> Inference (temps reel)
|      |      +-- Recommandation (cosine similarity)
|      |      +-- Scoring expert (PageRank+features)
|      |      +-- Prediction lacunes (collab.filt.)
|

```



```

|           +-- Classification doc (predict)           |
|                                                         |
+-----+

```

6.2 Détail des Algorithmes

Algorithme	Entrée	Technique	Sortie
Recommandation d'experts	Requête textuelle	TF-IDF + similarité cosinus	Top K experts triés par score
Scoring d'influence	Graphe Expert-Skill-Project	PageRank (d=0.85, 20 itérations)	Classement global des experts
Prédiction de lacunes	Matrice User-Skill	Filtrage collaboratif (cosinus)	Skills recommandées par expert
Compétences émergentes	Profils + dates d'embauche	Analyse de cohortes	Compétences en hausse/déclin
Prévision de demande	Projets + compétences	Projection linéaire	Demande à 6–24 mois
Classification docs	Contenu textuel	TF-IDF + Logistic Regression	Catégorie prédite
Collaboration inter-dept.	Skills par département	Analyse ensembliste	Paires de départements complémentaires

7. Pipeline Big Data

7.1 Simulateur Kafka

Le simulateur de pipeline Kafka simule l'ingestion d'événements temps réel dans l'organisation :

```

+-----+
|           SIMULATEUR KAFKA           |
+-----+
|
|  Evenements simules :                 |
|  +-- skill_added      (employe acquiert skill) |
|  +-- project_joined  (employe rejoint projet) |
|  +-- doc_published   (nouveau document publie) |
|  +-- collaboration   (interaction entre experts) |
|
|  Pipeline :                           |
|  Events -> Queue -> Processor -> Metrics |
|
|  Metriques :                           |
|  +-- events_per_second |
|  +-- total_processed   |
|  +-- error_rate        |

```

```
|    +-- queue_depth |
|                      |
+-----+
+-----+
```

7.2 Analyses Spark (Batch)

Les analyses Spark traitent l'ensemble des données JSONL en lot pour produire des rapports analytiques :

Analyse	Données en entrée	Résultat
Skill Analytics	skills.jsonl + employees.jsonl	Distribution et popularité des compétences
Expert Rankings	employees.jsonl + projects.jsonl	Classement des experts par score composite
Department Analytics	employees.jsonl	Statistiques par département (effectif, skills)
Document Analytics	documents.jsonl	Tendances des publications par thématique
Project Analytics	projects.jsonl	Statistiques par domaine, budget, statut

8. Architecture de Déploiement

8.1 Conteneurisation Docker

L'application est entièrement conteneurisée via **Docker Compose**, orchestrant trois services :

```
docker-compose.yml
|
+-- neo4j (Image: neo4j:5-community)
|   +-- Ports : 7474 (Browser UI), 7687 (Bolt)
|   +-- Volume : neo4j_data (persistant)
|   +-- Env : NEO4J_AUTH=neo4j/nexorapass
|   +-- Plugins : APOC
|   +-- Healthcheck : neo4j status (15s interval)
|
+-- backend (Build: backend/Dockerfile)
|   +-- Port : 8000
|   +-- Base : Python 3.11-slim
|   +-- Volume : ./data -> /app/data
|   +-- Env : NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD
|   +-- Depends : neo4j (condition: service_healthy)
|
+-- frontend (Build: frontend/Dockerfile)
    +-- Port : 3000 -> 80
    +-- Build multi-stage :
        |   +-- Stage 1 : Node 20 (npm build -> dist/)
```

```

|   +-- Stage 2 : Nginx Alpine (serve dist/)
+-- Nginx conf : SPA routing + API proxy
+-- Depends : backend

```

8.2 Configuration Nginx

Nginx joue un double rôle dans l'architecture :

1. **Serveur de fichiers statiques** : Sert les fichiers compilés React (HTML, JS, CSS) et gère le fallback SPA (toutes les routes non-fichier redirigées vers `index.html`).
2. **Proxy inverse** : Redirige les requêtes `/api/*` vers le backend FastAPI (port 8000), incluant le support WebSocket pour le chat temps réel.

8.3 Ordre de Démarrage et Dépendances

1. Neo4j démarre (healthcheck: 15s interval, max 5 retries)
 - |
 - v
2. Neo4j healthy -> Backend démarre
 - | (charge les JSONL, initialise les modèles ML)
 - |
 - v
3. Backend prêt -> Frontend démarre
 - | (Nginx écoute sur le port 3000)
 - |
 - v
4. Application opérationnelle

9. Description des Endpoints API

9.1 Endpoints Principaux

Méthode	Route	Rôle min.	Description
POST	/api/auth/register	Public	Inscription d'un nouvel utilisateur.
POST	/api/auth/login	Public	Authentification et obtention du JWT.
GET	/api/auth/users	Admin	Liste de tous les utilisateurs.
PUT	/api/auth/users/{id}/role	Admin	Modification du rôle d'un utilisateur.
GET	/api/experts	User	Liste des experts du graphe Neo4j.
GET	/api/experts/{id}	User	Détail d'un expert spécifique.
GET	/api/experts/search	User	Recherche multicritères d'experts.
GET	/api/graph	User	Données du graphe (nœuds + liens).

POST	/api/ai/recommend	User	Recommandation d'experts par IA.
GET	/api/ai/expert-rank	User	Classement PageRank des experts.
POST	/api/ai/skill-gaps	User	Prédiction des lacunes de compétences.
GET	/api/ai/emerging-skills	Manager	Compétences émergentes.
GET	/api/ai/future-demand	Manager	Prévisions de demande.
POST	/api/ai/classify-doc	User	Classification automatique de document.
POST	/api/ai/chat	User	Interaction avec le chatbot Veda.
GET	/api/kafka/status	Admin	Statut du pipeline Kafka.
POST	/api/kafka/start	Admin	Démarrage de la simulation Kafka.
GET	/api/bigdata/spark	Admin	Résultats des analyses Spark.

10. Sécurité et Authentification

10.1 Flux d'Authentification JWT

```

+-----+
|                FLUX AUTHENTIFICATION                |
+-----+
|
| 1. Client envoie POST /api/auth/login                 |
|   Body: { username, password }                       |
|
| 2. Backend verifie :                                  |
|   +-- Rate limiting (5 tentatives / 5 min)           |
|   +-- bcrypt.verify(password, hashed_pwd)            |
|
| 3. Si valide -> Generation JWT :                     |
|   { sub: "username", role: "admin",                 |
|     exp: now + 24h }                                 |
|   Signe avec cle secrete (HS256)                   |
|
| 4. Client stocke le token (localStorage)              |
|
| 5. Requetes suivantes :                              |
|   Header: Authorization: Bearer <token>             |
|   -> Backend decode, verifie role,                  |
|       autorise ou refuse (403)                      |
|
+-----+

```

10.2 Matrice de Contrôle d'Accès (RBAC)

Fonctionnalité	User	Manager	Admin
Explorer le graphe 3D	✓	✓	✓
Rechercher des experts	✓	✓	✓
Recommandations IA	✓	✓	✓
Chatbot Veda	✓	✓	✓
Team Builder	✓	✓	✓
Prévisions compétences		✓	✓
Compétences émergentes		✓	✓
Dashboard analytique		✓	✓
Gérer les utilisateurs			✓
Modifier les rôles			✓
Métriques Big Data			✓
Pipeline Kafka			✓

11. Conclusion

L'architecture de Nexora a été conçue pour offrir un équilibre optimal entre **performance**, **modularité** et **extensibilité**. Les choix technologiques — Neo4j pour la modélisation en graphe, FastAPI pour l'API performante, React pour l'interface réactive, Docker pour le déploiement reproductible — forment un ensemble cohérent et moderne.

Le processus de collecte des données, s'appuyant sur un générateur synthétique configurable, garantit la disponibilité de jeux de données réalistes et reproductibles pour alimenter l'ensemble des composants : graphe de connaissances, moteur ML et analyses Big Data.

Cette architecture est conçue pour évoluer : l'ajout de nouveaux connecteurs de données (SIRH, GitHub, Jira), de nouveaux algorithmes ML, ou le passage à un cluster Neo4j/Spark en production ne nécessite qu'une extension modulaire des composants existants, sans refonte structurelle. Cette approche « API-First » et conteneurisée assure la pérennité et la maintenabilité du système.